

CS-406 Mobile Application Development
BS (CE)-2k20
Complex Engineering Problem



Submitted By:

20-CE-001

Agha Abdullah

Department of Computer Engineering
HITEC University Taxila

Lab Instructor:

Dr. Muhammad Bilal

Problem Statement

A school management required you to design a "Student Management System" for android mobile to manage student's record. They want you to maintain student's registration number, name, parent phone

number, and address, and the management can search the students by their name or registration number and display list of student. The school management is not sure will the “Cloud” based system will be more appropriate or locally stored management system.

Develop two an android application one application store student’s record locally (using SQLite) and other cloud based (using Firestore database). Now compare both application in term of performance, security and usability.

Tools

Android Studio

Google Firestore

Student Management System (SQLite Version)

Code:

Student.java

First, we have to create a “Student” class to represent each entity in the database.

```

public class Student {

    private String registrationNumber;
    private String name;    private String
phoneNumber;
    private String address;

    public Student() {

    }

    public Student(String registrationNumber, String name, String phoneNumber, String address) {
this.registrationNumber = registrationNumber;
    this.name = name;
    this.phoneNumber = phoneNumber;
    this.address = address;
    }

    public String getRegistrationNumber() {
    return registrationNumber;
    }

    public String getName() {
return name;
    }

    public String getPhoneNumber() {
    return phoneNumber;
    }

    public String getAddress() {
return address;
    }
}

```

SQLiteHelper.java

The next step is to create a DB Helper class.

```

public class SQLiteHelper extends SQLiteOpenHelper {

    private static final String DATABASE_NAME = "student_database";    private
static final int DATABASE_VERSION = 1;

```

```

// Table and columns
private static final String TABLE_STUDENTS = "students"; private
static final String COLUMN_ID = "_id";
private static final String COLUMN_REGISTRATION_NUMBER = "registration_number"; private static
final String COLUMN_NAME = "name";
private static final String COLUMN_PHONE_NUMBER = "phone_number"; private
static final String COLUMN_ADDRESS = "address";

// Create table query
private static final String CREATE_TABLE_STUDENTS =
    "CREATE TABLE " + TABLE_STUDENTS + "(" +
        COLUMN_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, " +
        COLUMN_REGISTRATION_NUMBER + " TEXT, " +
        COLUMN_NAME + " TEXT, " +
        COLUMN_PHONE_NUMBER + " TEXT, " +
        COLUMN_ADDRESS + " TEXT" +
    ")";

public SQLiteHelper(Context context) {
    super(context, DATABASE_NAME, null, DATABASE_VERSION);
}

@Override public void
onCreate(SQLiteDatabase db) {
    db.execSQL(CREATE_TABLE_STUDENTS);
}

@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    // Implement if needed
}

```

StudentSQLiteHelper.java

Then we need to make a class to handle the **queries** such as **adding a student**, **searching a student** and getting all the student data from the database and add them to **Student class objects**.

```

public class StudentSQLiteHelper {

    private SQLiteDatabase database;

```

```

    public StudentSQLiteHelper(Context context) {
        SQLiteHelper dbHelper = new SQLiteHelper(context);
        database = dbHelper.getWritableDatabase();
    }

    public void addStudent(Student student) {
        ContentValues values = new ContentValues();
        values.put("registration_number", student.getRegistrationNumber());
        values.put("name", student.getName());
        values.put("phone_number", student.getPhoneNumber());
        values.put("address", student.getAddress());

        database.insert("students", null, values);
    }

    public List<Student> getAllStudents() {
        List<Student> studentList = new ArrayList<>();

        Cursor cursor = database.query("students", null, null, null, null, null, null);
        try {
            if (cursor != null && cursor.moveToFirst()) {
                do {
                    String registrationNumber =
                        cursor.getString(cursor.getColumnIndex("registration_number"));
                    String name = cursor.getString(cursor.getColumnIndex("name"));
                    String phoneNumber = cursor.getString(cursor.getColumnIndex("phone_number"));
                    String address = cursor.getString(cursor.getColumnIndex("address"));

                    Student student = new Student(registrationNumber, name, phoneNumber, address);
                    studentList.add(student);
                } while (cursor.moveToNext());
            } finally {
                if
                    (cursor != null) {
                        cursor.close();
                    }
            }

            return studentList;
        }

    }

    public List<Student> searchStudentsByName(String query) {
        List<Student> studentList = new ArrayList<>();

        // Query for students with matching name or registration number

```

```

String selection = "name LIKE ? OR registration_number LIKE ?";
String[] selectionArgs = {"%" + query + "%", "%" + query + "%"};

Cursor cursor = database.query("students", null, selection, selectionArgs, null, null, null);
try {
    if (cursor != null && cursor.moveToFirst()) {
do {
        String registrationNumber =
cursor.getString(cursor.getColumnIndex("registration_number"));
        String name = cursor.getString(cursor.getColumnIndex("name"));
        String phoneNumber = cursor.getString(cursor.getColumnIndex("phone_number"));
        String address = cursor.getString(cursor.getColumnIndex("address"));

        Student student = new Student(registrationNumber, name, phoneNumber, address);
        studentList.add(student);        } while (cursor.moveToNext());
    }
    } finally {        if
(cursor != null) {
    cursor.close();
    }
}

return studentList;
}

```

StudentAdapter.java

The StudentAdapter class is used to set the Student data into the UI Fields.

```

public class StudentAdapter extends ArrayAdapter<Student> {

    private Context context;
    private int resource;
    private ArrayList<Student> students;

    public StudentAdapter(@NonNull Context context, int resource, @NonNull ArrayList<Student>
students) {

```

```

        super(context, resource, students);
this.context = context;    this.resource
= resource;
        this.students = students;
    }

    @NonNull
    @Override
    public View getView(int position, @Nullable View convertView, @NonNull ViewGroup parent) {
        View listItem = convertView;

        if (listItem == null) {
            listItem = LayoutInflater.from(context).inflate(resource, parent, false);
        }

        Student currentStudent = students.get(position);

        TextView textViewRegistrationNumber =
listItem.findViewById(R.id.textViewRegistrationNumber);
        TextView textViewName = listItem.findViewById(R.id.textViewName);
        TextView textViewPhoneNumber = listItem.findViewById(R.id.textViewPhoneNumber);    TextView
textViewAddress = listItem.findViewById(R.id.textViewAddress);

        textViewRegistrationNumber.setText("Registration Number: " +
currentStudent.getRegistrationNumber());
        textViewName.setText("Name: " + currentStudent.getName());
        textViewPhoneNumber.setText("Phone Number: " + currentStudent.getPhoneNumber());
        textViewAddress.setText("Address: " + currentStudent.getAddress());

        return listItem;
    }
}

```

MainActivity.java

```

public class MainActivity extends AppCompatActivity {

    private EditText editTextRegistrationNumber, editTextName, editTextPhoneNumber,
editTextAddress;
    private Button btnAddStudent, btnSearchStudent;
    private ListView listViewStudents;    private
ArrayList<Student> studentList;
    private StudentAdapter studentAdapter;

    private StudentSQLiteHelper sqliteHelper;

```

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    editTextRegistrationNumber = findViewById(R.id.editTextRegistrationNumber);    editTextName
    = findViewById(R.id.editTextName);
    editTextPhoneNumber = findViewById(R.id.editTextPhoneNumber);    editTextAddress
    = findViewById(R.id.editTextAddress);

    btnAddStudent = findViewById(R.id.btnAddStudent);    btnSearchStudent
    = findViewById(R.id.btnSearchStudent);

    listViewStudents = findViewById(R.id.listViewStudents);
    studentList = new ArrayList<>();
    studentAdapter = new StudentAdapter(this, R.layout.list_item_student, studentList);
    listViewStudents.setAdapter(studentAdapter);

    sqliteHelper = new StudentSQLiteHelper(this);

    btnAddStudent.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            addStudentSQLite();
        }
    });

    btnSearchStudent.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            searchStudentSQLite();
        }
    });

    listViewStudents.setOnItemClickListener(new AdapterView.OnItemClickListener() {
        @Override
        public void onItemClick(AdapterView<?> adapterView, View view, int position, long id) {
            // Handle item click if needed
        }
    });

    // Load initial student list
    loadStudentListSQLite();
}

```

```

private void addStudentSQLite() {
    // Get user input
    String registrationNumber = editTextRegistrationNumber.getText().toString().trim();
    String name = editTextName.getText().toString().trim();
    String phoneNumber = editTextPhoneNumber.getText().toString().trim();
    String address = editTextAddress.getText().toString().trim();

    // Check if fields are not empty
    if (registrationNumber.isEmpty() || name.isEmpty() || phoneNumber.isEmpty() ||
address.isEmpty()) {
        Toast.makeText(this, "All fields are required", Toast.LENGTH_SHORT).show();
return;
    }

    // Create a new Student object
    Student student = new Student(registrationNumber, name, phoneNumber, address);

    // Add the student to SQLite
    sqliteHelper.addStudent(student);

    Toast.makeText(this, "Student added successfully", Toast.LENGTH_SHORT).show();
    clearInputFields();
    loadStudentListSQLite(); // Refresh the student list
}

private void searchStudentSQLite() {
    // Get user input for search
    String searchQuery = editTextName.getText().toString().trim();

    // Query SQLite for students matching the search query
    List<Student> searchResults = sqliteHelper.searchStudentsByName(searchQuery);

    studentList.clear();
studentList.addAll(searchResults);
    studentAdapter.notifyDataSetChanged();
}

private void loadStudentListSQLite() {
    // Query SQLite to load all students
    List<Student> allStudents = sqliteHelper.getAllStudents();

    studentList.clear();
studentList.addAll(allStudents);
    studentAdapter.notifyDataSetChanged();
}

```

```

    }

    private void clearInputFields() {
    editTextRegistrationNumber.setText("");
    editTextName.setText("");    editTextPhoneNumber.setText("");
    editTextAddress.setText("");
    }}

```

Layout Files:

list_item_student.xml

```

<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"    android:orientation="vertical"
    android:padding="10dp">

    <TextView
    android:id="@+id/textViewRegistrationNumber"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textStyle="bold"
        android:text="Registration Number: " />

    <TextView
        android:id="@+id/textViewName"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textStyle="bold"
        android:text="Name: " />

    <TextView
    android:id="@+id/textViewPhoneNumber"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textStyle="bold"
        android:text="Phone Number: " />

    <TextView
    android:id="@+id/textViewAddress"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
        android:textStyle="bold"
        android:text="Address: " />

</LinearLayout>

```

Activity_main.xml

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
```

```
    <EditText      android:id="@+id/editTextRegistrationNumber"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Registration Number" />
```

```
    <EditText
    android:id="@+id/editTextName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
        android:layout_below="@id/editTextRegistrationNumber"      android:hint="Student
    Name" />
```

```
    <EditText
        android:id="@+id/editTextPhoneNumber"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_below="@id/editTextName"      android:hint="Parent
    Phone Number"
        android:inputType="phone" />
```

```
    <EditText      android:id="@+id/editTextAddress"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_below="@id/editTextPhoneNumber"
        android:hint="Address" />
```

```
    <Button      android:id="@+id/btnAddStudent"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@id/editTextAddress"
    android:layout_marginLeft="150dp"
        android:text="Add Student" />
```

```
    <Button      android:id="@+id/btnSearchStudent"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@id/btnAddStudent"
    android:layout_marginLeft="140dp"
        android:text="Search Student" />
```

```
<ListView    android:id="@+id/listViewStudents"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:layout_below="@id/btnSearchStudent"
android:layout_marginTop="16dp" />

</RelativeLayout>
```

Code:

Student.java

```
public class Student {

    private String registrationNumber;
    private String name;    private String
    phoneNumber;
    private String address;

    // Default constructor required for Firestore
    public Student() {
    }

    public Student(String registrationNumber, String name, String phoneNumber, String address) {
    this.registrationNumber = registrationNumber;
        this.name = name;
        this.phoneNumber = phoneNumber;
        this.address = address;
    }

    // Getters and setters
    public String getRegistrationNumber() {
        return registrationNumber;
    }

    public void setRegistrationNumber(String registrationNumber) {
    this.registrationNumber = registrationNumber;
    }

    public String getName() {
    return name;
    }
```

```

    public void setName(String name) {
        this.name = name;
    }

    public String getPhoneNumber() {
        return phoneNumber;
    }
    public void setPhoneNumber(String phoneNumber) {
this.phoneNumber = phoneNumber;
    }

    public String getAddress() {
return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }
}

```

StudentAdapter.java

```

public class StudentAdapter extends ArrayAdapter<Student> {

    private int resourceLayout;
    private Context mContext;

    public StudentAdapter(Context context, int resource, List<Student> items) {
super(context, resource, items);    this.resourceLayout = resource;
        this.mContext = context;
    }

    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
View v = convertView;

        if (v == null) {
LayoutInflater vi;
            vi = LayoutInflater.from(mContext);
            v = vi.inflate(resourceLayout, null);
        }

        Student student = getItem(position);

        if (student != null) {

```

```

        TextView textViewName = v.findViewById(R.id.textViewName);
        TextView textViewRegistrationNumber = v.findViewById(R.id.textViewRegistrationNumber);
        TextView textViewPhoneNumber = v.findViewById(R.id.textViewPhoneNumber);
        TextView textViewAddress = v.findViewById(R.id.textViewAddress);

        if (textViewName != null) {
            textViewName.setText(student.getName());
        }

        if (textViewRegistrationNumber != null) {
            textViewRegistrationNumber.setText(student.getRegistrationNumber());
        }

        if (textViewPhoneNumber != null) {
            textViewPhoneNumber.setText("Phone: " + student.getPhoneNumber());
        }

        if (textViewAddress != null) {
            textViewAddress.setText("Address: " + student.getAddress());
        }
    }

    return v;
}

```

MainActivity.java

```

public class MainActivity extends AppCompatActivity {

    private EditText editTextRegistrationNumber, editTextName, editTextPhoneNumber,
    editTextAddress;
    private Button btnAddStudent, btnSearchStudent;
    private ListView listViewStudents; private
    ArrayList<Student> studentList;
    private StudentAdapter studentAdapter;

    private FirebaseFirestore firestore;
    private CollectionReference studentsCollection;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}

```

```

        editTextRegistrationNumber = findViewById(R.id.editTextRegistrationNumber);    editTextName
= findViewById(R.id.editTextName);
        editTextPhoneNumber = findViewById(R.id.editTextPhoneNumber);    editTextAddress
= findViewById(R.id.editTextAddress);

        btnAddStudent = findViewById(R.id.btnAddStudent);    btnSearchStudent
= findViewById(R.id.btnSearchStudent);    listViewStudents =
findViewById(R.id.listViewStudents);    studentList = new ArrayList<>();
        studentAdapter = new StudentAdapter(this, R.layout.list_item_student, studentList);
listViewStudents.setAdapter(studentAdapter);

        firestore = FirebaseFirestore.getInstance();
        studentsCollection = firestore.collection("students");

        btnAddStudent.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                addStudentFirestore();
            }
        });

        btnSearchStudent.setOnClickListener(new View.OnClickListener() {
            @Override    public void
onClick(View view) {
                searchStudentFirestore();
            }
        });

        listViewStudents.setOnItemClickListener(new AdapterView.OnItemClickListener() {
            @Override
            public void onItemClick(AdapterView<?> adapterView, View view, int position, long id) {
// Handle item click if needed
            }
        });

        // Load initial student list
loadStudentListFirestore();
    }

    private void addStudentFirestore() {
        // Get user input
        String registrationNumber = editTextRegistrationNumber.getText().toString().trim();
        String name = editTextName.getText().toString().trim();
        String phoneNumber = editTextPhoneNumber.getText().toString().trim();
        String address = editTextAddress.getText().toString().trim();

```

```

        // Check if fields are not empty
        if (registrationNumber.isEmpty() || name.isEmpty() || phoneNumber.isEmpty() ||
address.isEmpty()) {
            Toast.makeText(this, "All fields are required", Toast.LENGTH_SHORT).show();
return;
        }
        // Create a new Student object
        Student student = new Student(registrationNumber, name, phoneNumber, address);

        // Add the student to Firestore
        studentsCollection.document(registrationNumber)
            .set(student)
            .addOnCompleteListener(new OnCompleteListener<Void>() {
                @Override
                public void onComplete(@NonNull Task<Void> task) {
                    if (task.isSuccessful()) {
                        Toast.makeText(MainActivity.this, "Student added successfully",
Toast.LENGTH_SHORT).show();
                        clearInputFields();
                        loadStudentListFirestore(); // Refresh the student list
                    } else {
                        Toast.makeText(MainActivity.this, "Failed to add student",
Toast.LENGTH_SHORT).show();
                    }
                }
            });
    }

    private void searchStudentFirestore() {
        // Get user input for search
        String searchQuery = editTextName.getText().toString().trim();

        // Query Firestore for students matching the search query
        studentsCollection.whereEqualTo("name", searchQuery)
            .get()
            .addOnCompleteListener(new OnCompleteListener<QuerySnapshot>() {
                @Override
                public void onComplete(@NonNull Task<QuerySnapshot> task) {
                    if (task.isSuccessful()) {
                        studentList.clear();
                        for (QueryDocumentSnapshot document : task.getResult()) {
                            Student student = document.toObject(Student.class);
                            studentList.add(student);
                        }
                    }
                }
            });
    }

```

```

        studentAdapter.notifyDataSetChanged();
    } else {
        Toast.makeText(MainActivity.this, "Error searching for students",
Toast.LENGTH_SHORT).show();
    }
}
});
}

private void loadStudentListFirestore() {
// Query Firestore to load all students
studentsCollection.get()
    .addOnCompleteListener(new OnCompleteListener<QuerySnapshot>() {
        @Override
        public void onComplete(@NonNull Task<QuerySnapshot> task) {
if (task.isSuccessful()) {
            studentList.clear();
            for (QueryDocumentSnapshot document : task.getResult()) {
Student student = document.toObject(Student.class);
                studentList.add(student);
            }
            studentAdapter.notifyDataSetChanged();
        } else {
            Toast.makeText(MainActivity.this, "Error loading student list",
Toast.LENGTH_SHORT).show();
        }
    }
});
}

private void clearInputFields() {
editTextRegistrationNumber.setText("");
editTextName.setText("");    editTextPhoneNumber.setText("");
    editTextAddress.setText("");
}}

```

Comparison and Evaluation:

Performance Comparison:

SQLite-Based Application:

1. **Local Storage:** SQLite stores data locally on the device, which can result in faster read and write operations compared to network-based solutions.
2. **Limited Scalability:** The performance is generally good for small to medium-sized datasets. However, as the dataset grows, performance might degrade.

